

Contents

Keep Me Ready — Founder Technical Blueprint v1	2
Table of Contents	2
1. Project Purpose	2
2. Current App Architecture	3
3. Folder and File Structure	4
4. Key Components and What They Do	6
DrillClient.tsx	6
LessonCard.tsx	6
ScoreDisplay.tsx	6
LessonReview.tsx	6
DashboardView.tsx	6
LearningJournal.tsx	7
InstructorDashboard.tsx	7
NavUser.tsx	7
5. Supabase Setup	7
Project creation	7
Auth configuration	7
Running migrations	7
6. Database Tables and Migrations	8
drill_sessions (created in 001, extended in 002)	8
drill_question_answers (created in 002)	8
Migration files	8
7. Authentication Flow	8
Sign-in sequence	9
Session persistence	9
Admin authentication	9
8. Admin Dashboard Logic	9
Data sources	9
The 9 metrics	10
Service role client	10
Graceful degradation	10
9. Vercel Deployment Process	10
Initial deployment	10
Subsequent deployments	11
Triggering a manual redeploy	11
Git remote	11
10. Environment Variables	11
11. Current Learner Flow	11
Anonymous learner (no account)	11
Authenticated learner (signed in)	12
Question bank	12
Topic rotation	12
12. Current Instructor/Admin Flow	12
13. Recent Features Implemented	13
Authentication fixes (June 2026)	13
Lesson Review System	13

Compare Perspectives (self-check)	13
Learning Journal	13
Instructor Dashboard	14
Dark mode heading contrast	14
14. Known Limitations	14
Authentication	14
Data and persistence	14
Analytics	14
Architecture	15
15. Future Roadmap	15
Near-term (next 1–3 sessions)	15
Medium-term	15
Long-term	15

Keep Me Ready — Founder Technical Blueprint v1

Prepared: June 2026

Version: 1.0

Repository: [git@github.com:tshatiqua-bit/keep-me-ready.git](https://github.com:tshatiqua-bit/keep-me-ready.git)

Live URL: <https://keep-me-ready.vercel.app>

Table of Contents

1. Project Purpose	
2. Current App Architecture	
3. Folder and File Structure	
4. Key Components and What They Do	
5. Supabase Setup	
6. Database Tables and Migrations	
7. Authentication Flow	
8. Admin Dashboard Logic	
9. Vercel Deployment Process	
10. Environment Variables	
11. Current Learner Flow	
12. Current Instructor/Admin Flow	
13. Recent Features Implemented	
14. Known Limitations	
15. Future Roadmap	

1. Project Purpose

Keep Me Ready is a daily bookkeeping refresher platform designed for job seekers, bookkeeping students, and early-career professionals who need to stay sharp between jobs, certifications, or client engagements.

The core philosophy is that professional competence is built through consistent, low-friction daily repetition — not cramming. Every design and product decision reflects three principles:

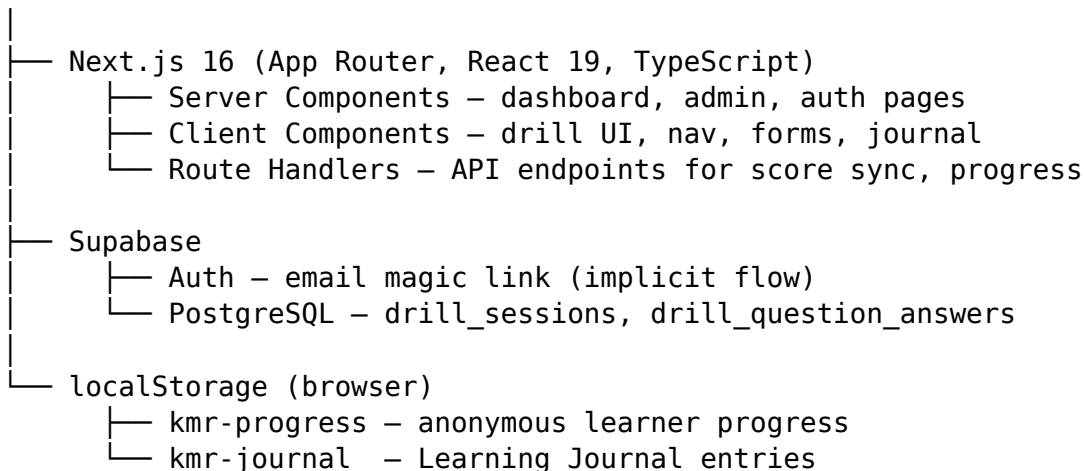
- **Practice, not testing.** Drills are coaching moments, not assessments.
- **Reflection builds mastery.** Learners are encouraged to compare perspectives and document their evolving thinking.
- **Progress compounds.** Small daily habits, tracked over time, produce lasting professional confidence.

Primary audience: Bookkeepers, accounting assistants, and small-business owners who want to stay current with foundational concepts across debits and credits, accounts, financial statements, journal entries, bank reconciliation, and payroll.

2. Current App Architecture

Keep Me Ready is a **Next.js 16 App Router** application deployed on **Vercel**, backed by **Supabase** for authentication and PostgreSQL storage.

Browser



Key architectural decisions:

- **App Router with Server Components** for pages that need server-side data (dashboard, admin). Client Components only where interactivity or browser APIs are required.
- **Dual data path:** Logged-in users get data from Supabase; anonymous users get data from localStorage. The UI handles both transparently.
- **Auth uses Supabase implicit flow.** The magic link delivers hash-fragment tokens (#access_token=...) that are processed client-side by createBrowserClient. The callback page listens for onAuthStateChange rather than exchanging a server-side code.
- **Middleware via proxy.ts** (Next.js 16 convention) refreshes Supabase sessions on every request to prevent expiry during active use.
- **Admin gate is env-var-based** — no roles table. The ADMIN_EMAIL environment variable is compared server-side against the authenticated user's email.

3. Folder and File Structure

keep-me-ready/

```
|
|
|--- app/                                # Next.js App Router pages and routes
|   |--- layout.tsx                     # Root layout – nav header, footer
|   |--- page.tsx                      # Homepage / landing
|   |--- globals.css                   # Tailwind v4 import, global styles
|   |
|   |--- admin/
|       |--- page.tsx                  # Instructor Dashboard (admin-
only server component)
|
|   |--- api/
|       |--- scores/route.ts           # POST – saves drill session to Supabase
|       |--- progress/route.ts        # GET – fetches user's sessions from Supabase
|
|   |--- auth/
|       |--- callback/page.tsx         # Handles magic link return (client component)
|       |--- login/page.tsx           # Login page (magic link form)
|       |--- logout/route.ts          # GET – signs user out, redirects to /
|
|   |--- dashboard/
|       |--- page.tsx                  # Learner dashboard (server component, passes data to Dashboard
|
|   |--- drill/
|       |--- page.tsx                  # Drill page (renders DrillClient)
|
|--- components/
|   |--- DashboardView.tsx             # Client component – renders full dashboard UI
|   |--- HomeStats.tsx                 # Anonymous stats on homepage
|   |--- LoginForm.tsx                 # Magic link email form
|   |--- NavUser.tsx                  # Auth-aware nav (Sign in / email + Sign out)
|   |--- SaveProgressPrompt.tsx        # Prompt for anonymous users to create an account
|   |
|   |--- admin/
|       |--- InstructorDashboard.tsx    # Admin analytics UI (9 metrics)
|
|   |--- dashboard/
|       |--- LearningJournal.tsx       # Learning Journal section (reads kmr-
journal from localStorage)
|       |--- ProgressStats.tsx         # 4-stat grid (drills, questions, correct, accuracy)
|       |--- SessionHistory.tsx        # Recent drill sessions list
|       |--- StreakBadge.tsx           # Current streak display
|
|   |--- drill/
```

```

├── AnswerOption.tsx    # Single answer button with correct/incorrect state
├── DrillClient.tsx     # Main drill state machine ("use client")
├── DrillProgress.tsx   # Question counter and live score bar
├── Explanation.tsx     # Post-answer coaching explanation
├── LessonCard.tsx      # Pre-drill lesson overview card
├── LessonReview.tsx    # Post-drill per-question review with coaching
├── QuestionCard.tsx    # Renders the question text
├── ScoreDisplay.tsx    # Results screen – score, reflection, Learning Journal
├── data/
│   ├── questions/
│   │   ├── beginner.json    # 72 questions across 6 categories (active)
│   │   ├── intermediate.json # Placeholder – not yet in use
│   │   └── advanced.json    # Placeholder – not yet in use
│   └── lib/
│       ├── questions/
│       │   ├── selector.ts    # Picks 10 questions for a drill session
│       │   └── topics.ts      # Topic metadata, lesson content, reflection prompts
│       ├── supabase/
│       │   └── admin.ts        # Service-role client (bypasses RLS –
server-only)
│       │   ├── client.ts      # Browser client (createBrowserClient)
│       │   └── server.ts      # Server client (createServerClient + cookies)
│       └── utils/
│           ├── progress.ts    # localStorage read/write for anonymous progress
│           └── streak.ts      # Computes current streak from session dates
├── supabase/
│   ├── migrations/
│   │   ├── 001_init.sql      # Creates drill_sessions + RLS policies
│   │   └── 002_analytics.sql # Adds category/started_at, creates drill_question_answers
├── types/
│   └── index.ts              # Shared TypeScript types (Question, DrillSession, UserProgress)
├── proxy.ts                  # Next.js 16 middleware (session refresh on every request)
├── next.config.ts            # Next.js configuration
├── .env.local                # Local secrets – never committed
├── .env.local.example        # Template for required environment variables
└── package.json

```

4. Key Components and What They Do

DrillClient.tsx

The heart of the drill experience. A client-side state machine managing five phases:

Phase	Description
lesson	Shows the LessonCard — topic overview, why it matters, common confusion
answering	Active question — learner selects an answer
revealed	Answer shown — correct/incorrect state, coaching explanation
completed	Score screen (ScoreDisplay) with reflection and Learning Journal
review	Full LessonReview of all 10 questions with per-option coaching

State tracked: `selectedAnswers` (array of 10), `score`, `currentIndex`, `phase`, `drill-StartedAt`. On completion, syncs to Supabase via `/api/scores`.

LessonCard.tsx

Displayed before the drill begins. Shows topic title, lesson text, why it matters, common confusion, and an encouragement message. Learner taps “Start Drill” to enter the answering phase.

ScoreDisplay.tsx

Results screen after all 10 questions. Shows score, a contextual coaching message, a “What to review next” card, and the “One Reflection to Sit With” card. The reflection card contains: - **Compare Perspectives** — expandable section with 4–5 key idea bullets and a model response - **Learning Journal** — optional textarea; saves entry to `localStorage["kmr-journal"]` on “Save This Insight”

LessonReview.tsx

Walks through all 10 drill questions post-completion. Each card shows the question, all answer options (correct in green, learner’s wrong answer in red), expandable wrong-answer coaching notes per option, and a “Key Lesson Takeaways” block at the bottom.

DashboardView.tsx

Client component that merges server-side Supabase data (logged-in users) with local-Storage data (anonymous users). Renders `StreakBadge`, `ProgressStats`, `SessionHistory`, and `LearningJournal`. Detects which data source to use and handles the brief loading state for anonymous users.

LearningJournal.tsx

Reads `localStorage["kmr-journal"]`, renders entries newest-first. Shows date, topic label, and insight text per entry. Displays topic filter pills automatically when entries span more than one topic. Includes empty state guiding the learner to “Compare Perspectives” after a drill. Architecture is ready for milestone celebrations at 25/50/100 entries.

InstructorDashboard.tsx

Admin-only analytics UI. Pure server component — no client state. Displays 9 metrics across 4 rows: stat cards, improvement/completion time, weekly trend bar chart, and most difficult topics / most missed questions panels.

NavUser.tsx

Client component in the root layout. Subscribes to `supabase.auth.onAuthStateChange` and shows either a “Sign in” link or the user’s email with a “Sign out” button.

5. Supabase Setup

Project creation

1. Create a new project at supabase.com
2. Note the **Project URL** and **anon key** from Settings → Data API
3. Note the **service role key** from Settings → Data API (keep strictly server-side)

Auth configuration

1. Go to **Authentication → URL Configuration**
2. Set **Site URL** to `https://keep-me-ready.vercel.app`
3. Add `https://keep-me-ready.vercel.app/auth/callback` to **Redirect URLs**
4. Email magic links are enabled by default — no additional provider setup required for MVP

Running migrations

Both SQL migration files must be run manually in **Supabase → SQL Editor**:

1. Run `supabase/migrations/001_init.sql` — creates the `drill_sessions` table with RLS
2. Run `supabase/migrations/002_analytics.sql` — adds analytics columns and the `drill_question_answers` table

Order matters. Migration 002 references the table created in 001.

6. Database Tables and Migrations

drill_sessions (created in 001, extended in 002)

Column	Type	Description
id	UUID	Primary key, auto-generated
user_id	UUID	Foreign key → auth.users
date	DATE	Drill date (YYYY-MM-DD)
score	INTEGER	Number of correct answers
total	INTEGER	Total questions in the drill
completed_at	TIMESTAMPTZ	Timestamp of completion
category	TEXT	Topic category slug (<i>added in 002</i>)
started_at	TIMESTAMPTZ	Drill start time (<i>added in 002</i>)

RLS policies: Users can only SELECT and INSERT their own rows (`auth.uid() = user_id`).

drill_question_answers (created in 002)

Column	Type	Description
id	UUID	Primary key, auto-generated
session_id	UUID	Foreign key → drill_sessions
user_id	UUID	Foreign key → auth.users
question_id	TEXT	Stable question identifier from JSON
category	TEXT	Topic category slug
was_correct	BOOLEAN	Whether the learner answered correctly
answered_at	TIMESTAMPTZ	Timestamp of the answer

RLS policies: Users can only INSERT and SELECT their own rows.

Migration files

supabase/migrations/001_init.sql — baseline schema

supabase/migrations/002_analytics.sql — analytics extension

Both files use IF NOT EXISTS guards and are safe to re-run.

7. Authentication Flow

Keep Me Ready uses **Supabase email magic links** with **implicit flow** (hash-fragment tokens).

Sign-in sequence

1. Learner enters email on `/auth/login`
2. `LoginForm` calls `supabase.auth.signInWithOtp({ email, options: { emailRedirectTo: `${location.origin}/auth/callback` } })`
3. Supabase sends a magic link email
4. Learner clicks the link → browser navigates to Supabase's verification endpoint
5. Supabase verifies the token and redirects to:
`https://keep-me-ready.vercel.app/auth/callback#access_token=...&refresh_token=...`
6. `app/auth/callback/page.tsx` loads (client component)
7. `createBrowserClient` detects the hash tokens automatically
8. `onAuthStateChange` fires with event `"SIGNED_IN"` and a valid session
9. The callback page calls `router.replace("/dashboard")`
10. `NavUser` detects the new session and updates the header

Session persistence

- Sessions are stored in browser cookies by `@supabase/ssr`
- `proxy.ts` (middleware) calls `supabase.auth.getUser()` on every request to refresh the session token before it expires
- Logging out hits `/auth/logout` which calls `supabase.auth.signOut()` and redirects to `/`

Admin authentication

The `/admin` page is a server component that:

1. Calls `supabase.auth.getUser()` using the server-side client (reads session from request cookies)
2. If no user → redirect(`"/auth/login"`)
3. Compares `user.email` to `process.env.ADMIN_EMAIL`
4. If no match → renders an "Access Denied" page showing which email is signed in (helps diagnose misconfiguration without leaking the admin email value)
5. If match → renders `InstructorDashboard` with full analytics data

8. Admin Dashboard Logic

The Instructor Dashboard is accessible at `/admin` and is protected by the `ADMIN_EMAIL` environment variable.

Data sources

- **`drill_sessions`** — queried via the Supabase service role client (bypasses RLS, sees all users' data)
- **`drill_question_answers`** — queried for per-question miss rate and topic difficulty
- **`auth.users`** — user count and last-seen data

The 9 metrics

Metric	Source
Total Users	Count of rows in <code>auth.users</code>
Active Today	Sessions with <code>date = today</code>
Drills Completed	Count of all <code>drill_sessions</code> rows
Average Score	Mean of <code>score</code> / <code>total</code> across all sessions
30-Day Active Rate	Users with a session in the last 30 days ÷ total users
Average Improvement	Per-user: first-half session avg vs. second-half session avg
Average Completion Time	Mean of <code>completed_at</code> - <code>started_at</code> (requires migration 002)
Most Difficult Topics	Categories ranked by lowest correct rate (requires migration 002)
Most Missed Questions	Questions ranked by highest incorrect rate, min 3 attempts (requires migration 002)

Service role client

`lib/supabase/admin.ts` creates a Supabase client with the service role key. This client is **never used on the client side** — only in server components and route handlers. It bypasses all RLS policies, allowing cross-user analytics.

Graceful degradation

If migration 002 has not been applied, the admin page detects the absence of `drill_question_answers` data and displays an amber notice. Core metrics (total users, drills completed, average score) work from migration 001 alone.

9. Vercel Deployment Process

Initial deployment

1. Push the repository to GitHub
2. Import the repository in the Vercel dashboard
3. Vercel auto-detects Next.js — no build configuration needed
4. Add all required environment variables (see Section 10) under **Settings → Environment Variables**
5. Set environment scope to **Production** (and optionally Preview)
6. Deploy

Subsequent deployments

Every push to the main branch triggers an automatic Vercel deployment. No manual steps required.

Triggering a manual redeploy

To pick up new environment variables without a code change, run:

```
git commit --allow-empty -m "chore: trigger redeploy" && git push origin main
```

Or use the Vercel dashboard: **Deployments** →  → **Redeploy**.

Git remote

All pushes use SSH:

```
git remote set-url origin git@github.com:tshatiqua-bit/keep-me-ready.git
```

10. Environment Variables

The following environment variables must be set in Vercel under **Settings** → **Environment Variables**. No secret values are documented here.

Variable	Where used	Required
NEXT_PUBLIC_SUPABASE_URL	All Supabase clients (browser + server)	Yes
NEXT_PUBLIC_SUPABASE_ANON_KEY	All Supabase clients (browser + server)	Yes
SUPABASE_SERVICE_ROLE_KEY	Admin client only (lib/supabase/admin.ts) — server-side only, never exposed to browser	Yes
ADMIN_EMAIL	/admin page auth gate — server-side only	Yes

Local development: Copy `.env.local.example` to `.env.local` and fill in real values. This file is gitignored and must never be committed.

NEXT_PUBLIC_ prefixed variables are embedded in the browser bundle at build time. All others remain server-side only.

11. Current Learner Flow

Anonymous learner (no account)

Homepage → Start Drill

- Lesson Card (topic overview)
- 10 Questions (multiple choice, true/false, scenario)
- Score Screen
 - Compare Perspectives (key ideas + model response)
 - Learning Journal (optional – saves to localStorage)
 - Review Lesson (per-question coaching)
- Dashboard (stats from localStorage)

Progress (streak, accuracy, session history) is stored in `localStorage["kmr-progress"]`. A “Sign in to save your progress” prompt appears on the score screen.

Authenticated learner (signed in)

Same flow, with these additions: - Drill completion syncs to Supabase via `POST /api/scores` - Dashboard stats are server-rendered from `drill_sessions` - Learning Journal entries remain in `localStorage` (Supabase sync planned for a future phase) - Session history persists across devices

Question bank

- **72 beginner questions** across 6 categories
- 3 question types: multiple choice (4 options), true/false (2 options), scenario (4 options)
- 10 questions selected per drill via `lib/questions/selector.ts`
- Today’s topic rotates daily based on `dayOfYear % 6` — learners always know what to expect
- Each question includes: explanation, per-option coaching notes (`optionExplanations`), and is tagged with category, type, and level

Topic rotation

#	Category	Focus
1	debits_credits	Debits & Credits — The Foundation
2	accounts	Chart of Accounts
3	financial_statements	Financial Statements
4	journal_entries	Journal Entries
5	bank_reconciliation	Bank Reconciliation
6	payroll	Payroll

12. Current Instructor/Admin Flow

- `/admin` (requires `ADMIN_EMAIL` match)
- Server fetches all `drill_sessions` via service role client
 - Computes 9 metrics in TypeScript
 - Renders `InstructorDashboard`

- Stat cards (5 primary metrics)
- Improvement + completion time cards
- Weekly trend bar chart (last 8 weeks)
- Most Difficult Topics (ranked by avg correct rate)
- Most Missed Questions (ranked by miss rate, min 3 attempts)

The dashboard is read-only. All data is computed at request time — there is no caching layer currently. The page re-fetches fresh data on every visit.

If the admin visits before migration 002 is applied, an amber warning banner explains which features are inactive and what SQL to run.

13. Recent Features Implemented

Authentication fixes (June 2026)

- Replaced the original server-side Route Handler callback with a client-side page (`app/auth/callback/page.tsx`) that handles both PKCE code exchange and implicit flow hash tokens via `onAuthStateChange`
- Fixed admin gate to redirect to `/auth/login` (not `/`) when not logged in, and show a diagnostic “Access Denied” page with the signed-in email when the email doesn’t match

Lesson Review System

- Post-drill review shows all 10 questions with correct/incorrect indicators
- Expandable per-option coaching notes explain why each wrong answer is incorrect and what misconception it represents
- “Key Lesson Takeaways” block at the bottom of the review

Compare Perspectives (self-check)

- Expandable section on the score screen
- Each topic has 4–5 key idea bullets and a plain-English model response
- Framed as perspective comparison, not correct/incorrect grading

Learning Journal

- Optional reflection textarea on the score screen
- Prompt: “How has your thinking shifted after comparing perspectives?”
- “Save This Insight” saves to `localStorage["kmr-journal"]`
- Entry shape: `{ id, category, conceptId, topicTitle, prompt, text, date, savedAt }`
- `conceptId` field future-proofs analytics and mastery tracking
- Confirmation: “☐ Added to your Learning Journal / Every reflection you save becomes part of your professional growth.”
- **Dashboard section:** `LearningJournal.tsx` renders entries newest-first with topic filter pills

Instructor Dashboard

- 9 analytics metrics across 4 visual rows
- Service role Supabase client bypasses RLS for cross-user queries
- Admin gate via ADMIN_EMAIL environment variable
- Graceful degradation when migration 002 is not applied

Dark mode heading contrast

- All primary headings use dark:text-slate-400 or dark:text-slate-300 throughout the app
 - Input fields use explicit text-slate-900 dark:text-slate-100 with matching dark backgrounds
-

14. Known Limitations

Authentication

- **Magic link only.** Google, Apple, and Microsoft OAuth are deferred post-MVP. The login form currently shows only the email/magic link flow.
- **Implicit flow (not PKCE).** Supabase delivers tokens as hash fragments. The callback handles this correctly, but it means auth cannot be completed server-side (requires JavaScript in the browser).
- **Single admin account.** The ADMIN_EMAIL gate supports only one administrator email. A roles table would be needed to support multiple admins.

Data and persistence

- **Learning Journal is localStorage-only.** Journal entries do not sync across devices and are lost if the browser's localStorage is cleared. Supabase sync is planned for a future phase.
- **Anonymous progress is device-specific.** localStorage data cannot be merged with a Supabase account retroactively without a deliberate migration step.
- **Question bank is beginner-only.** Intermediate and advanced JSON files exist but are empty placeholders.

Analytics

- **Admin metrics require migration 002.** Per-question analytics, topic difficulty, and completion time are unavailable until both migrations are applied.
- **No caching on the admin dashboard.** Every visit re-fetches all session data. This is acceptable at low user volumes but will need pagination or caching at scale.
- **No real-time updates.** The admin dashboard is a static snapshot at the time of page load.

Architecture

- **No test suite.** There are no automated unit, integration, or end-to-end tests. All testing is manual.
 - **No error monitoring.** There is no Sentry, Datadog, or similar integration. Server errors surface only in Vercel function logs.
 - **Question selection is random, not adaptive.** Questions are selected randomly from the day's category. There is no spaced repetition or performance-based question weighting yet.
-

15. Future Roadmap

Near-term (next 1-3 sessions)

- **Learning Journal — Supabase sync.** Persist journal entries to a new `learning_journal` table in Supabase. Preserve all existing `localStorage` entries on first sync. Enable cross-device access.
- **Learning Journal milestone celebrations.** Quiet celebrations at 25, 50, 100, and 250 insights captured. Emphasis on accumulated understanding rather than streaks or scores.
- **Learning Journal dashboard timeline.** Organize the journal view by date or topic. Show growth over time visually.

Medium-term

- **Google / Apple / Microsoft OAuth.** Reduce friction for learners who prefer social sign-in.
- **Intermediate and advanced question banks.** 72+ questions per difficulty level, selectable by learner experience level.
- **Adaptive question selection.** Weight questions toward areas where the learner has historically underperformed. Implement spaced repetition for missed concepts.
- **Mastery tracking per concept.** Use `conceptId` (already stored in journal entries and question metadata) to show per-topic mastery levels on the dashboard.
- **Streak protection and reminders.** Email or push reminders before a streak expires. Daily digest of tomorrow's topic.

Long-term

- **Multi-admin support.** Replace `ADMIN_EMAIL` with a `user_roles` table supporting multiple admin and instructor accounts.
- **Cohort / class management.** Instructors can create groups, assign topics, and view aggregate performance for their cohort.
- **Content management.** Admin UI for adding and editing questions, topics, and coaching notes without a code deploy.
- **Mobile app.** React Native or PWA wrapper for offline drill capability.

- **Certification prep tracks.** Curated question sequences aligned to QuickBooks, Xero, or bookkeeping certification exams.
- **Public API.** Allow third-party accounting educators to integrate Keep Me Ready drills into their own platforms.

Keep Me Ready — Practice makes permanent.